

Rapport de révision — Copilot IA PROBAR

LLM • Provider • RAG • Agents • Intent Detection • Tool Calling • MCP • RBAC • PendingAction • Calendrier

Fiche basée sur les audits et résultats techniques fournis. Les éléments non confirmés doivent être revérifiés dans le dépôt avant le mémoire final.

1. Architecture globale

Le Copilot est intégré à Flutter côté frontend et à Node.js côté backend. Le parcours général est : utilisateur → interface Copilot → API → AI Copilot Bridge → orchestrateur → détection d'intention → Agent → LLM/RAG/Tools → services métier → PostgreSQL → réponse Flutter.

Utilisateur → Flutter → API → AI Copilot Bridge → Orchestrateur → Intent → Agent → LLM / RAG / Tool Calling → Services → PostgreSQL → ResponseBuilder → Flutter

2. LLM

LLM signifie Large Language Model. Il sert à comprendre les demandes en langage naturel, produire des réponses et participer au Tool Calling selon l'Agent. Il n'est pas l'autorité de sécurité : identité, rôles et scope sont contrôlés côté serveur.

3. Provider LLM — Ollama

Les logs fournis montrent explicitement le provider Ollama. Un timeout de 180000 ms et un suivi des tokens prompt/completion ont été observés.

```
[ai:llm:ollama] chat request failed - timeout of 180000ms exceeded
token usage: prompt=2631 completion=18
```

Le modèle exact et ses paramètres complets doivent être confirmés dans la configuration actuelle avant d'être écrits comme faits définitifs.

4. Intent Detection

Le système utilise un catalogue d'intentions, un scoring/confidence et des overrides ciblés. Les travaux ont confirmé notamment capability_audit, projects, commercial_contacts et por_promesh. Le résultat de l'intention détermine l'Agent.

Message → IntentDetector → scoring/confidence → override éventuel → intent → Agent

5. Multi-Agent

L'architecture est spécialisée par domaine. Les travaux ont confirmé notamment ProjectAgent, CommercialAgent, MeetingAgent, AutomationAgent, PorPromeshAgent et HRAgent.

Agent	Domaine
ProjectAgent	Projets
CommercialAgent	Contacts et relances
MeetingAgent	Réunions / calendrier
AutomationAgent	Automatisations
PorPromeshAgent	POR PROMESH
HRAgent	RH

6. Prompt Engineering

Les Agents utilisent des instructions système pour définir rôle, contraintes, Tools et comportement attendu. Le prompt améliore le comportement du LLM mais ne remplace jamais le RBAC.

7. Tool Calling

Le Tool Calling transforme une demande naturelle en opération métier structurée. Le Tool est soumis à validation et autorisation avant d'atteindre le service et la base.

User → Agent → LLM → ToolCallingChain → Tool → Joi/Validation → Permission/RBAC → Service → DB → résultat → réponse

8. MCP

Les audits identifient une couche MCP autour des Tools : registre, schémas, appels et validation. MCP structure l'exposition des capacités ; les permissions restent contrôlées par le backend.

9. RAG

RAG signifie Retrieval-Augmented Generation. Le système récupère des informations pertinentes dans une source documentaire et les ajoute au contexte envoyé au LLM.

Documents → ingestion → parsing/chunking → embedding → recherche → retrieval → contexte → prompt → LLM → réponse/citations

Les informations fournies mentionnent un embedding par hashing de 256 dimensions et un stockage en mémoire. Ces détails doivent être revérifiés dans le code actuel avant utilisation comme spécification finale.

10. Embeddings et Vector Store

Un embedding représente un texte sous forme numérique afin de permettre une comparaison de proximité. Le vector store sert à rechercher les représentations pertinentes. Les détails exacts de persistance et de similarité doivent être confirmés dans le dépôt.

11. RAG vs Tool Calling

RAG	Tool Calling
Récupération de contexte documentaire	Exécution d'une capacité métier
Passages/contexte	Résultat structuré
Connaissance documentaire	Données/services applicatifs
Exemple : documentation	Exemple : projets / POR PROMESH

12. requireToolUse

Pour POR PROMESH, une formulation réelle pouvait produire final_answer sans appeler list_por_promesh malgré un routage correct. requireToolUse: true a été ajouté à PorPromeshAgent pour empêcher cette réponse finale non fondée lorsqu'un Tool est requis.

13. RBAC et sécurité

Principe clé : LLM input → Tool → actingUser JWT → scope serveur → DB. Le LLM ne décide pas des permissions. Les audits ont testé notamment ownerId, userId et createdBy contre les mécanismes d'ownership.

14. PendingAction

PendingAction permet de proposer une action puis de demander confirmation. Le contrat normalisé est {id, steps:[{tool,input,reason}]}. Flutter affiche cette structure ; l'exécution réelle relit l'action stockée côté serveur.

Demande → proposition → PendingAction → confirmation → consume(conversationId) → Tool → DB

15. Relance commerciale

Le flux de relance résout contact et commercial côté serveur, crée une PendingAction puis exécute le Tool après confirmation. La synchronisation peut créer une Task CRM et une CalendarEventSync puis tenter Google Calendar.

16. MeetingAgent et CalendarEventSync

L'entité copilot_meeting a été ajoutée à l'ENUM PostgreSQL via la migration 20260815120000-add-copilot-meeting-to-calendar-event-syncs-enum.js. Les tests ciblés ont ensuite été rapportés comme 9/9 passés.

Avant	project_action ; commercial_contact_relance
Après	project_action ; commercial_contact_relance ; copilot_meeting
Tests ciblés	9/9 après migration

17. Tests

Les missions ont couvert tests unitaires, intégration, DB réelle, RBAC, anti-fuite, confirmation, annulation et idempotence. La baseline communiquée comporte 21 échecs MFA préexistants.

18. Les 10 idées essentielles

1. LLM = compréhension et génération. 2. Ollama = provider observé. 3. Intent Detection = choix du domaine. 4. Agents = spécialisation métier. 5. Tools = opérations contrôlées. 6. MCP = structuration des Tools. 7. RAG = contexte documentaire. 8. RBAC = sécurité serveur. 9. PendingAction = confirmation. 10. Tests = validation du comportement.

19. Questions du jury

Question	Réponse
Pourquoi un LLM ?	Comprendre les formulations naturelles et orchestrer l'assistance.
Quel provider ?	Ollama apparaît dans les logs fournis.
RAG ou Tool Calling ?	RAG récupère du contexte ; Tool Calling exécute une capacité.
Le LLM décide-t-il du RBAC ?	Non, le serveur fait foi.
Pourquoi des Agents ?	Spécialiser les capacités par domaine.
Pourquoi Joi ?	Valider les entrées des Tools.
Pourquoi PendingAction ?	Confirmer une action avant exécution.
Pourquoi requireToolUse ?	Empêcher final_answer sans Tool lorsqu'il est requis.
Comment éviter une double exécution ?	Consume et idempotence côté serveur.
Pourquoi CalendarEventSync ?	Suivre la synchronisation avec le calendrier.

20. Points à vérifier avant soutenance

Revérifier dans le dépôt : modèle LLM exact, paramètres du provider, nombre exact d'Agents/Tools, implémentation précise du RAG/embeddings/vector store, versions des dépendances et derniers résultats de tests. Ne pas transformer une hypothèse en fait.